TFG · SONDA PASIVA DE DETECCIÓN PQC · TAREA 2

Análisis de key_share en cliente y servidor

Bloques 2, 3 y 4 de la sonda: cabecera TCP, reensamblado y lectura de TLS con clasificación

Pablo Marugán Bastante · GITST, EPS-UAM · Tutor: Jorge E. López de Vergara | **04/11/2026** | Tutoría del miércoles 04/11/2026

1. Conclusión principal

> RESULTADO
>
> **La sonda ya reconstruye las conexiones TCP, lee el ClientHello, el HelloRetryRequest y el ServerHello, y dice si el `key_share` lleva grupos PQ o híbridos**, tanto en el cliente como en el servidor. Clasifica cada conexión, cliente y servidor con la tabla de la tarea 1 (`grupos_pqc.csv`, cargada al arrancar). Se ha validado con Wireshark/tshark sobre tres trazas y con AddressSanitizer sobre 1500 handshakes corruptos, sin errores de memoria.

2. Arquitectura: de paquete a veredicto

Bloque	Qué hace	Por qué hace falta
1 (tutoría 1)	Ethernet, VLAN e IPv4	Llegar a la cabecera de transporte
2	Cabecera TCP, filtro de puerto (`-p`, por defecto 443)	Separar cabecera y datos; la longitud de la cabecera TCP es variable (opciones)
3	Tabla de flujos y reensamblado	El ClientHello con clave PQ no cabe en un segmento (tarea 1: 1750 B en 1460 + 290)
4	Lectura de TLS y clasificación	Extraer `supported_groups` y `key_share` y aplicar la tabla de grupos

3. Bloque 2: cabecera TCP

Struct empaquetado `tcp_hdr` con el mismo patrón que `ip_hdr`. La longitud de la cabecera se lee del *data offset* (4 bits altos del byte 12, en palabras de 32 bits) y nunca se asume 20. Los datos se miden con la longitud de IP, no con `caplen`, por el relleno de Ethernet. Comprobado línea a línea contra Wireshark con números de secuencia absolutos:

```
#1  10.0.0.2:54321 -> 140.82.113.4:443  [SYN]      seq=1000 ack=0     len=0
#2  140.82.113.4:443 -> 10.0.0.2:54321  [SYN,ACK]  seq=5000 ack=1001  len=0
#4  10.0.0.2:54321 -> 140.82.113.4:443  [PSH,ACK]  seq=1001 ack=5001  len=1460  <- empieza registro TLS
(1750 B)
#5  10.0.0.2:54321 -> 140.82.113.4:443  [PSH,ACK]  seq=2461 ack=5001  len=290
#6  140.82.113.4:443 -> 10.0.0.2:54321  [PSH,ACK]  seq=5001 ack=2751  len=127
```

El paquete 5 continúa exactamente donde acabó el 4 (1001 + 1460 = 2461) y el servidor confirma hasta el 2750 (2461 + 290 = 2751). El paquete 5 no empieza por `16 03`: es la continuación del mensaje, y analizado por separado no se entiende.

4. Bloque 3: flujos y reensamblado

- **Tabla de flujos:** hash de 4096 cubetas con listas enlazadas. La clave (IP:puerto, IP:puerto) se guarda en orden canónico para que ida y vuelta caigan en el mismo flujo.
- **Colocación por posición:** cada segmento va a `posición = seq − (ISN + 1)` en un buffer por sentido, con un mapa de bits de bytes recibidos. Resuelve a la vez los segmentos desordenados y las retransmisiones.
- **Registro completo:** la longitud se lee en los 5 primeros bytes del registro; cuando los bytes contiguos la alcanzan, el registro está entero.
- **Sin SYN** (captura a mitad de conexión): solo se acepta si el primer segmento empieza por `16 03`.

Prueba con `trazas/reensamblado_desordenado.pcap`

Paq.	Trozo del ClientHello	Sonda	Wireshark 4.6.9
4	1º (0–599)	Contiguos: 600	—
5	1º repetido	Retransmisión	TCP Retransmission
6	3º (1200–1411)	Desordenado: hueco en 600–1199	Previous segment not captured
7	2º (600–1199)	**ClientHello completo** (1412 B, 3 segmentos)	TCP Retransmission (incorrecto: aporta bytes nuevos)

Wireshark etiqueta el paquete 7 como retransmisión por una heurística (su `seq` es menor que el mayor visto). La sonda lo decide byte a byte con el mapa: no es copia, sino el trozo que faltaba. Para validar contra Wireshark/tshark debe estar activa la preferencia TCP *Reassemble out-of-order segments*.

5. Bloque 4: lectura de TLS y clasificación

- **Tabla de grupos externa:** `grupos_pqc.csv` se carga al arrancar (opción `-g`). Actualizar el listado no requiere recompilar (reivindicación 8). GREASE se detecta por patrón.
- **Lector con control de límites:** cada lectura comprueba antes que quedan bytes. Un campo de longitud falso detiene la lectura en vez de salirse del buffer.
- **ClientHello:** extensiones 10 (`supported_groups`, lo que el cliente **sabe**), 51 (`key_share`, para qué grupos **manda clave** y de cuántos bytes), 43 (versiones) y 0 (SNI).
- **ServerHello / HelloRetryRequest:** mismo formato; el HRR se reconoce por su *random* fijo (RFC 8446 §4.1.3) y su `key_share` solo trae el grupo pedido. Tras un HRR, la sonda salta el ChangeCipherSpec de compatibilidad y lee el segundo ClientHello.
- **Comprobación de longitudes:** cada clave se compara con la longitud esperada del CSV (`(ok)` o `(ESPERADO n)`).

Reglas de clasificación

Elemento	Regla
Conexión	**Segura** si se negoció en TLS 1.3 un grupo PQ o híbrido; **insegura** en otro caso
Cliente	Por lo que sabe hacer: **inseguro** sin PQ · **potencialmente inseguro** con PQ y también clásico · **seguro** solo PQ
Servidor	**Fuerza PQ** (lo pidió con HRR) · **soporta PQ** (lo eligió) · **inseguro** si tenía la clave PQ, o el cliente ofrecía TLS 1.3, y eligió algo sin PQ · **indeterminado** si el cliente no le dio la opción

Prueba con `trazas/casos_clasificacion.pcap` (un caso por situación)

SNI	Cliente manda	Servidor	Conexión	Servidor clasificado
1-pq-elegido	X25519MLKEM768 + x25519	X25519MLKEM768	Segura	Soporta PQ
2-cliente-clasico	x25519 (no sabe PQ)	x25519	Insegura	Indeterminado
3-hrr-fuerza-pq	**x25519; tras HRR, X25519MLKEM768**	**HRR → X25519MLKEM768**	**Segura**	**Fuerza PQ**
4-rechaza-clave-pq	X25519MLKEM768 + x25519	x25519	Insegura	Inseguro
5-no-fuerza-pq	**x25519 (anuncia PQ)**	**x25519**	**Insegura**	**Indeterminado**
6-servidor-tls12	X25519MLKEM768 + x25519	TLS 1.2	Insegura	Inseguro

Cliente: inseguro en el caso 2 y potencialmente inseguro en el resto. Con la traza de la tarea 1 (`clienthello_pqc.pcap`) la sonda da exactamente la clasificación razonada a mano: conexión insegura, cliente potencialmente inseguro y servidor inseguro. El caso 3

se ha comprobado en Wireshark con el filtro `ip.addr == 104.16.3.3 && tls`: Client Hello → Hello Retry Request (pide 4588 = X25519MLKEM768) → Change Cipher Spec + Client Hello → Server Hello.

> ★ **PARA COMENTAR EN LA TUTORÍA**
>
> En los casos 3 y 5 **el cliente hace exactamente lo mismo**: anuncia X25519MLKEM768 y solo manda clave x25519. Lo que cambia es el servidor: uno pide PQ con un HelloRetryRequest y el otro acepta clásico. Es el comportamiento del **45 % medido en la tutoría 1**, y la sonda ya lo distingue de forma pasiva. El caso 5 deja al servidor **indeterminado**; resolverlo exige **acumular estado por servidor**: si en otra conexión elige PQ, pasa a *potencialmente inseguro* (sabe PQ pero acepta clásico). Es el siguiente paso del código.

6. Validación y limitaciones

Prueba	Resultado
`clienthello_pqc.pcap` (tarea 1)	Reensamblado 1750 B en 2 segmentos y clasificación iguales a Wireshark y al análisis manual
`reensamblado_desordenado.pcap`	1 desordenado y 1 retransmisión detectados; ClientHello de 1412 B en 3 segmentos
`casos_clasificacion.pcap`	Las 6 situaciones clasificadas según la tabla
AddressSanitizer + UBSan, 1500 handshakes mutados	Sin errores de memoria; 126 mensajes rechazados como malformados

Pendiente: estado acumulado por servidor; IPv6 (el bloque 1 aún lo descarta); caducidad de flujos para captura en vivo larga; tráfico real con PQ (capturar Chrome por la interfaz Wi-Fi de Windows con Npcap). Las trazas de prueba son sintéticas, generadas con Scapy (tarea 3).